IIT JODHPUR

Feb'26 Minor Examination: EEL3090 Embedded Systems $\boxed{Set - B}$

Guidelines (Total time: 120 minutes, Maximum Marks: 25):

- Please read question paper very carefully. **NO clarification is required in any question.** In case of any doubt, assume and state that in your answer.
- **Please mention Set Number in your answer-sheet.**

---

1. Your friend is hired as an embedded systems engineer in a safety-critical industrial automation company. He asks you to help design a multi-level alarm generation system using an STM32 microcontroller. This kit has the following specifications: a) The STM32 system clock (HCLK) is configured to 72 MHz. b) SysTick is a 24-bit down counter, and it can use either: i) HCLK (72 MHz), or ii) HCLK/8 as its clock source. The system designed by you must generate the following: a) A Level-1 Warning Alarm every 400 ms, b) A Level-2 Critical Alarm every 3 seconds Determine whether it is possible to directly generate a 3-second interrupt using SysTick without any involvement from software (in the form of software counters etc.)? If possible, calculate: required reload value? If not, explain why ? [2 + 1]

2. You are designing a power system diagnostic unit for a deep-space probe onboard computer. The system uses an internal ADC to monitor battery voltage via an analog input pin. The microcontroller has: 10- bit ADC, One analog input channel connected to battery voltage divider, GPIO pins configurable as digital input/output, ADC End-Of-Conversion (EOC) flag, ADC conversion complete interrupt and 60 MHz system clock. The ADC conversion time is 12 μs. The system does the following: start an ADC conversion every 8 ms, detect when the conversion is complete, toggle a GPIO debug pin HIGH when conversion starts, toggle the GPIO LOW when conversion result is read. There are two methods to achieve the above sequence of tasks: either through continuous monitoring of respective flags or through interrupts. Draw the software/application flow (main loop and functions) for both these methods? Mention the merits/demerits of both these methods. [2 + 2]

3. You are working as a system architect at *JodhpurEmbedded*, where you are redesigning a real-time text-matching embedded platform intended for secure industrial terminals. The system receives 8-bit ASCII characters from a keyboard at unpredictable intervals. The system must perform real-time pattern matching against a stored reference string (maximum length = 64 characters) and transmit only the matched characters to a serial LCD terminal with UART-style transfer that follows 8 data bits, no parity, 1 stop bit (8N1 format) and baud-rate deployed is known as 115200. The microcontroller runs at a clock frequency of 48 MHz and includes these components: UART peripheral, Interrupt controller, DMA controller, Timer modules, GPIO, and Internal SRAM. Your system must satisfy the following additional constraints: i) The keyboard may generate bursts of characters at up to 3 kHz. ii) The LCD cannot tolerate buffer overrun, and iii) The CPU must remain in low-power sleep mode whenever possible
   - Derive the UART baud rate divisor required to generate 115200 bps from a 48 MHz clock ? [1]
   - Considering 8N1 transmission format, calculate the maximum number of characters per second that can be transmitted to the LCD? [1]
   - Determine whether the system can safely handle a 3 kHz burst from the keyboard without data loss? If answer to above is NO, suggest one remedy? If answer is yes, write N/A? [1 + 1]
   - Draw a detailed block-level architecture of the complete system, clearly identifying the internal microcontroller components used. [1]

4. For the purpose of interrupt generation using hardware timers, you are approached by your friend working at a semiconductor company *Infineon*.
   (a) For a 72 MHz system clock, calculate the appropriate prescaler value and compare (auto-reload) value required to generate a 250 ms periodic interrupt using a 16-bit timer. [1.5]
   (b) Can this 16-bit hardware timer directly generate a 120-second delay ? Show all necessary calculations and clearly explain your reasoning. [1.5]

5. You are recruited as an embedded firmware engineer at a high-security defense hardware company and are tasked with developing a secure multi-mode control interface using pure bare-metal, register-level programming on an STM32 microcontroller (strictly no HAL, CubeMX, middleware, or driver libraries). The system operates with an HCLK frequency of 72 MHz.
   Three push buttons are connected to GPIOC pins PC3, PC4, and PC5 in pull-up configuration, while two LEDs are connected to GPIOD pins PD8 and PD9 configured as push-pull outputs. The firmware must implement three operating modes entirely through polling logic inside the main loop. In Normal Mode, pressing the button on PC3 toggles LED1 (PD8), and pressing the button on PC4 toggles LED2 (PD9). In Secure Mode, the button on PC3 is allowed to toggle LED1 only if the button on PC4 is pressed within a 150 ms time window, while pressing the button on PC5 immediately turns off both LEDs and any invalid input sequences must be ignored. In Lockdown Mode, which is activated when all three buttons are detected as pressed simultaneously, the two LEDs must blink alternately at approximately 8 Hz, and the system should exit this mode only if the button on PC5 is continuously held for more than 2 seconds.

   You are required to describe in detail how you would configure the necessary RCC and GPIO registers, implement robust software debouncing strictly using polling techniques, detect simultaneous button conditions reliably, generate approximate timing delays using calibrated software delay loops based on the 72 MHz clock, and organize the main control logic as a deterministic finite-state machine. Additionally, explain how you would avoid race conditions in shared variables and ensure stable GPIO transitions in a fully polling-based bare-metal system implementation. [3.5]

6. You are appointed as an embedded systems consultant at a defense electronics startup tasked with developing a signal emulation module for testing and calibration of battlefield Electronic Intelligence (ELINT) receivers. Instead of intercepting radar pulses, your system must generate programmable PWM-style waveforms that mimic the timing characteristics of hostile radar emissions. The generated waveform should allow controlled adjustment of pulse width, duty cycle, and pulse repetition interval (PRI) so that different radar signatures can be simulated for laboratory testing. You decide to use an STM32 Nucleo development board operating at 16 MHz and utilize its hardware timer in PWM output mode to synthesize the required square-wave signal. Explain, in a step-by-step manner (with relevant block and timing diagrams), how you would configure the timer's prescaler, auto-reload register (ARR), and capture/compare register (CCR) to generate PWM waveforms with specified ON-time and repetition interval. Also describe how varying duty cycle and frequency parameters can help emulate different classes of radar systems for testing and validation purposes. [3]

7. Convert below ARM assembly into equivalent C/C++ sub-routine (lr is link register, RSB is reverse subtract, RSBS refers to reverse subtract with updates of the condition flags based on the result)? [3.5]

```
        PUSH   {r4, r5, lr}
        MOV    r2, #0
        MOV    r3, #0
loop:
        CMP    r3, r1
        BGE    done
        LDR    r4, [r0, r3, LSL #2]
        CMP    r4, #0
        BGE    positive
        RSBS   r4, r4, #0
positive:
        CMP    r4, #10
        BLE    skip
        ADD    r2, r2, r4
skip:
        ADD    r3, r3, #1
        B      loop
done:
        MOV    r0, r2
        POP    {r4, r5, lr}
        BX     lr
```